

1 Introduction

1.1 Background

TDA

Topological Data Analysis (TDA) is a framework that uses algebraic topology to analyze data and extract topological descriptors [5]. Persistence homology is the main technique of TDA [10, 23], and tracks the birth and death of topological features using filtration [10], resulting in persistence barcodes and derived features [6]. Such outputs, like persistence barcodes and diagrams, provide compact summaries of topological features that can be used in statistical analysis or machine learning [34]. For instance, persistence is used in material science to detect cavities and channels in porous materials [17], and in cosmology to describe and compare different Cosmic Web shape models [25]. Persistence homology is robust to noise in input data [8]. This stability makes TDA suitable for a wide range of applications, including denoising signals [16], detecting anomalies [33], analyzing medical data [28], segmenting images [7], and extracting features for machine learning models [3].

TDA can be applied to many modalities, such as time series, images, and graphs. For 1-dimensional arrays like time series, there are two main ways to compute the persistence; either directly via sublevel-set filtration [11], or indirectly by first converting the time series into a point cloud via time-delay embedding [29], then applying a distance-based filtration, such as Vietoris-Rips [13]. While the delay embeddings approach is useful in revealing the dynamic system’s underlying topology, it introduces more hyperparameters (i.e., embedding dimension and delay), increasing computational cost, and produces results that are sensitive to the choice of these hyperparameters [30].

For 2-dimensional arrays like grayscale images, the persistence can be computed via sublevel-set filtration. A sublevel-set filtration [11] models the array as a discretized continuous field, sweeping through data from lowest to highest value, like water filling the structure, to track how connected components and loops emerge and merge. Since the 2D array is formatted like a coordinate grid, it is logical to embed a cubical complex [14] onto the array. This complex utilizes the grid structure to make highly efficient data structures, and maps pixels to vertices, pixel pairs to edges, and pixel blocks to 2D faces. To assign filtration values to these edges and faces, we apply lower-star filtration [26], assigns each higher-dimensional cell the maximum intensity value of its defining boundary vertices. This setup eliminates geometric preprocessing and enables efficient memory allocation.

1.2 Streaming

Real-life data often arrives as data streams, where data observations are continuously received over time. While this enables timely analysis of changing phenomena, it prevents access to the full dataset, and requires online algorithms to match, having strict memory and computational requirements [2]. Examples of such include stock and medical data for the 1D case, while the 2D case includes images and climate data [32]. Given the increasing prevalence of streaming data, there is growing interest in computing the persistence of incoming data on-the-fly. This allows the analysis of rapidly evolving systems, like weather maps or stock prices, and has been investigated in the context of scalable computation over large and continuous data sources [23]. In principle, online computation of persistent homology should be faster and lighter than recomputing from scratch after each data arrival, since only new updates are processed. However, in practice, accurately computing the persistence requires information of the entire array, such as keypoints, which depends on the old and newly arriving data.

Motivation

Streaming data is an underexplored topic in TDA, despite their prevalence in modern applications. Existing persistence homology pipelines tackle static datasets and request recomputation when data updates, causing large computational overhead. While TDA is used in critical applications like hurricane analysis [31], it is usually applied in a static setting; when climate data is updated, the computation is rerun from scratch. In rapidly evolving scenarios that requires equally rapid responses, such inefficiencies are critical limitations. Also, many methods focus on point cloud representations and simplicial complexes, such as Vietoris-Rips, while structured 1D/2D array data is less addressed. This limits applicability in low-compute environments, such as individual computers, where efficiency is crucial. This work addresses such challenges by designing a light and efficient method for computing persistent homology for row-wise streaming 2D data. Our method is based on sub-level set filtration over cubical complexes, allowing incremental updates without full recomputation. Due to the localized memory footprint and single-pass streaming architecture, our method scales with available computational resources.

1.3 Related Works

There have been recent works to apply persistence computation for data streams. Moitra et al. [20] apply persistent homology to streaming point cloud data using Vietoris-Rips filtration, where their main contribution is an online summarization framework for memory-efficient persistence on data streams. In a further work [19], they used a sliding window approach for streaming, which better suits transient

changes. In both works, their method relies on the Vietoris-Rips complex, ruling it out as an interesting approach for this paper.

Cultrera di Montesano et al. [9] develop a method to compute the sublevel set persistence of a 1D time series that gets updated, which can be adapted for a streaming scenario, by using a dynamic tree-based structure they call the banana tree. However their approach is exclusively for 1D, which is outside our scope.

Nigmatov and Morozov [22] propose an algorithm for distributed computation of persistence using sublevel-set filtration on cubical complexes. However, their paper targets massive computing environments, evidenced by their benchmarks on the Perlmutter supercomputer and validating Amdahl’s law [1]. Their approach splits a data array across many nodes, because the dataset is too large to fit on a single machine, then each piece is processed on a node, after which all pieces are synchronized across the network to resolve global features; this approach is similar to but not exactly streaming. In addition, we are more interested in applying the streaming scenario to a more reachable audience, by achieving similar accuracy and scaling on a single-node computer, while also having the capacity to scale the streaming pipeline.

Libraries like Gudhi [18] and CubicalRipser [15] perform Vietoris-Rips and cubical sublevel-set filtrations, respectively. CubicalRipser is based on the popular C++-based library Ripser [4], though the latter only does Vietoris-Rips filtration, which does not directly work on time series and arrays without extractions. Gudhi works for multiple dimensions, while CubicalRipser works for up to 4 dimensions. Dionysus [21] is a library that computes persistent homology over filtrations of simplicial complexes, like Vietoris–Rips, rather than cubical complexes.

2 Method

2.1 Nomenclature

Persistent homology quantifies the emergence (birth) and disappearance (death) of topological features, doing so via filtration [10, 5]. These features are characterized by Betti numbers β_k , which count the number of independent k -dimensional holes [10]. For an n -dimensional dataset, homology groups H_k are computed for $k = 0, 1, \dots, n - 1$, and the birth and death times of the Betti components are measured. Betti numbers have geometric interpretations: β_0/H_0 is the number of connected components, β_1/H_1 is the number of 1-dimensional holes or loops, β_2/H_2 is the number of 2-dimensional voids or cavities, and higher-order Betti numbers represent the analog to holes in higher dimensions [12].

For the 1D/2D data considered in this work, the topological structures are as follows:

given a dataset with N points, let points u and v be vertices such that $u, v \in V$, where V is the set of vertices, and e is an edge that connects two vertices such that $e = (u, v) \in E$, where E is the set of edges. A face f is a filled 2D region bound by edges, and a loop L is a closed cycle of connected edges that is not filled by, or the boundary of, any face. Connected components H_0 correspond to connected vertices and edges, while loops H_1 correspond to closed cycles of edges that are not filled by faces. By tracking the birth and death of these features during the filtration process, we can discern meaningful structure from topological noise. Features that persist over a long time are more significant than short-lived ones [5].

3 Our Method

As previously mentioned, our method computes the persistence of a 1D/2D array using sublevel set filtration, using a cubical complex. This complex treats the array like a grid, and treats points like pixels. Thus, edges are exclusively horizontal and vertical lines, meaning points are only connected to their 4 direct neighbors, no diagonals. In sublevel set filtration, an edge appears only after both its endpoint vertices appear, so an edge’s birth time is the maximum of its vertices $\max(u, v)$. This also implies that a face appears when only all its boundary pixels appear, so its birth time is $\max(u_1, u_2, u_3, u_4)$.

Our implementation is in Python, using Torch/NumPy and Numba, and draws heavily from the ‘watershed’ analogy used for image processing [27]; given an array, and following the watershed analogy, where water pours from the top and fills basins (demarcated by minima), and basins merge when a bridge joins them (demarcated by maxima).

To track updates with streams, our method maintains a flat Union-Find parent/birth array tracker. This tracker uses parent pointers to identify the root vertex or face of a topological basin, allowing the system to quickly resolve global connectivity changes, without storing the underlying geometry.

Saddle points correspond to grid edges where components merge or split. For H_0 , births occur at local minima vertices, while deaths occur during the ascending sweep when an edge acts as a saddle connecting two basins, killing the younger component via the Elder Rule [24]. For H_1 , use descending co-sweep as tracker: an edge acts as a 1D saddle when it closes a spatial loop between two face components to mark its birth, and the loop dies when filled at the maximum filtration value of the enclosed faces.

3.1 Algorithm

Our method is outlined below, where steps 4-6 deal with streaming:

1. **Component births:** Compute exact birth times for vertices (pixels), edges, and faces across the grid using local coordinate maxima. **Output:** flat arrays of evaluated structural values `pix_vals`, `edge_vals`, `face_vals`.
2. **Forward H_0 sweep:** Execute a forward Union-Find pass over the vertices and edges, sorted from lowest to highest. Using the Elder Rule [24], components (H_0) merge when connected by an edge, recording the death of the younger component. **Output:** populated 0D birth-death tracking matrix (`h0_out`) and a vertex root-tracking array (`parent`).
3. **Backward H_1 sweep:** Sweep backward through the edges sorted from highest to lowest value, using an independent Union-Find tracker on the faces, to record births and deaths of 1D loops (H_1). **Output:** populated 1D birth-death tracking matrix (`h1_pairs`) and a face root-tracking array (`parent_f`).
4. **Stage incoming stream:** receive incoming array chunk from stream and extract its pixel, edge, and face values using local coordinate maxima (like Step 1) into pre-allocated memory blocks, to bypass slow resizing loops. **Output:** staged chunk-tracking matrices (`stream_h0`, `stream_h1`) packed into memory.
5. **Interface stitching:** process and update the Union-Find pointer-tracking arrays strictly across the shared boundary rows where data chunks meet. This resolves global topological features across chunks while removing expired chunk data from memory. **Output:** locally updated, global-connected pointer states in the `parent` and `parent_f` arrays.
6. **Diagram computing:** filter and pair the accumulated birth-death matrices, discarding invalid pairs (i.e., where death occurs before birth). **Output:** Final pruned coordinate matrices representing the final topological diagrams (`h0`, `h1`)

The vertices (pixels) u and v delimit the endpoints of each structural grid edge e , where e_{val} denotes the edge's filtration value. Elements f_1 and f_2 denote adjacent grid faces, `parent` is a lookup array, and `root` is the function used to find the scalar basin roots (r_u , r_v , and r_f) tracked by the Union-Find structures, while b and d explicitly denote the final compiled birth and death coordinates.

Algorithm 1: Streaming Basin Forest

In: Grid G , boundary row S **Out:** H_0, H_1

▷ 1. Extraction & Staging

1 : Valuate facets via `max()` filtration
2 : Stage flat structures: `stream_h0, stream_h1`

▷ 2. Forward H_0 Sweep

3 : `parent` $\leftarrow -1$; `birth_val` $\leftarrow 0$
4 : **for** $(e_{val}, u, v) \in \text{sort_asc}(\text{stream_h0})$ **do**
5 : $r_u, r_v \leftarrow \text{Root}(u), \text{Root}(v)$
6 : **if** $r_u \neq r_v$ **then**
7 : Record $(\text{birth}[\text{victim}], e_{val})$ to `h0_out` via `min()`
8 : `parent[victim]` $\leftarrow \text{survivor}$
9 : **end for**

▷ 3. Backward H_1 Sweep & Stitching

10 : `parent_f` $\leftarrow -1$; `birth_val_f[ext]` $\leftarrow \infty$
11 : **for** $(e_{val}, f_1, f_2) \in \text{sort_desc}(\text{stream_h1})$ **do**
12 : $r_{f1}, r_{f2} \leftarrow \text{Root}(f_1), \text{Root}(f_2)$
13 : **if** $r_{f1} \neq r_{f2}$ **then**
14 : **if** $\text{victim} \neq \text{ext}$ **then** Record $(e_{val}, \text{birth_f}[\text{victim}])$
15 : `parent_f[victim]` $\leftarrow \text{survivor}$ ▷ Stitches boundary seam
16 : **end for**

▷ 4. Compile Diagrams

17 : $H_0 \leftarrow \{(b, d) \in \text{h0_out} \mid d > b\} \cup \{(\text{min}(\text{pix}), \infty)\}$
18 : $H_1 \leftarrow \{(b, d) \in \text{h1_pairs} \mid d > b\}$
19 : **return** H_0, H_1

3.2 Results

We compare our method against the state-of-the-art libraries CubicalRipser and Gudhi, for wall-clock time and peak memory usage.

We run the experiments on an Apple MacBook M4 Pro, with a 14-core CPU (10 performance and 4 efficiency cores), and 48 GB unified memory, using a single-machine for all experiments. We run the experiments 3 times, and show the average and standard deviation.

We generate a square matrix randomly and apply uniform smoothing using an aver-

Table 1: Benchmark results of our method (non-streaming) and standard TDA Python libraries

Datapoints	Method	Time [s]	Peak Memory [MiB]
$N = 1 \times 10^6$	Gudhi	1.118 ± 0.020	878.09 ± 0.47
	CubicalRipser	0.073 ± 0.003	778.36 ± 0.00
	Ours	0.206 ± 0.011	1081.00 ± 0.00
$N = 4 \times 10^6$	Gudhi	5.417 ± 0.062	1698.59 ± 77.57
	CubicalRipser	0.380 ± 0.009	2016.94 ± 0.00
	Ours	1.042 ± 0.013	1795.13 ± 1.61
$N = 1.6 \times 10^7$	Gudhi	26.715 ± 0.182	5256.77 ± 165.13
	CubicalRipser	1.762 ± 0.120	5311.30 ± 0.63
	Ours	4.827 ± 0.013	5662.15 ± 0.01
$N = 6.4 \times 10^7$	Gudhi	130.709 ± 1.789	17889.10 ± 484.07
	CubicalRipser	7.828 ± 1.067	20990.78 ± 67.04
	Ours	19.556 ± 0.280	20575.23 ± 995.83

Table 2: Results for our method’s streaming scenario, for a varying number of datapoints and streaming chunks

Datapoints	# Chunks	Total Time [s]	Peak Memory [MiB]
1×10^6	2	0.428 ± 0.001	925.13 ± 1.60
	5	0.441 ± 0.008	930.59 ± 1.10
	10	0.455 ± 0.008	997.75 ± 4.78
4×10^6	2	2.349 ± 0.025	2116.00 ± 7.62
	5	2.252 ± 0.005	2040.40 ± 61.42
	10	2.271 ± 0.010	2398.06 ± 21.49
1.6×10^7	2	11.045 ± 0.020	5618.46 ± 29.91
	5	11.002 ± 0.016	6257.73 ± 714.68
	10	11.492 ± 0.065	5883.88 ± 524.83

age pooling filter, varying the number of pixels on each side with 1000, 2000, 4000, 8000 pixels per side.

References

- [1] Gene M Amdahl. “Validity of the single processor approach to achieving large scale computing capabilities”. In: *Proceedings of the April 18-20, 1967, spring joint computer conference*. 1967, pp. 483–485.
- [2] Brian Babcock et al. “Models and issues in data stream systems”. In: *Proceedings of the twenty-first ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 2002, pp. 1–16.
- [3] Danielle Barnes, Luis Polanco, and Jose A Perea. “A comparative study of machine learning methods for persistence diagrams”. In: *Frontiers in Artificial Intelligence* 4 (2021), p. 681174.
- [4] Ulrich Bauer. “Ripser: efficient computation of Vietoris–Rips persistence barcodes”. In: *Journal of Applied and Computational Topology* 5.3 (2021), pp. 391–423.

- [5] Gunnar Carlsson. “Topology and data”. In: *Bulletin of the American Mathematical Society* 46.2 (2009), pp. 255–308.
- [6] Gunnar Carlsson et al. “Persistence barcodes for shapes”. In: *Proceedings of the 2004 Eurographics/ACM SIGGRAPH symposium on Geometry processing*. 2004, pp. 124–135.
- [7] James R Clough et al. “A topological loss function for deep-learning based image segmentation using persistent homology”. In: *IEEE transactions on pattern analysis and machine intelligence* 44.12 (2020), pp. 8766–8778.
- [8] David Cohen-Steiner, Herbert Edelsbrunner, and John Harer. “Stability of persistence diagrams”. In: *Proceedings of the twenty-first annual symposium on Computational geometry*. 2005, pp. 263–271.
- [9] Sebastiano Cultrera di Montesano et al. “Dynamically maintaining the persistent homology of time series”. In: *Proceedings of the 2024 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM. 2024, pp. 243–295.
- [10] Herbert Edelsbrunner and John Harer. *Computational topology: an introduction*. American Mathematical Soc., 2010.
- [11] Edelsbrunner, Letscher, and Zomorodian. “Topological persistence and simplification”. In: *Discrete & computational geometry* 28.4 (2002), pp. 511–533.
- [12] Allen Hatcher. *Algebraic topology*. Cambridge University Press, 2005.
- [13] Jean-Claude Hausmann. “On the Vietoris–Rips Complexes and a Cohomology Theory”. In: *Prospects in topology: proceedings of a conference in honor of William Browder*. 138. Princeton University Press. 1995, p. 175.
- [14] Tomasz Kaczynski, Konstantin Mischaikow, and Marian Mrozek. *Computational Homology*. Vol. 157. Applied Mathematical Sciences. Springer New York, 2004.
- [15] Shizuo Kaji, Takeki Sudo, and Kazushi Ahara. “Cubical ripser: Software for computing persistent homology of image and volume data”. In: *arXiv preprint arXiv:2005.12692* (2020).
- [16] Jennifer Kloeke and Gunnar Carlsson. “Topological de-noising: Strengthening the topological signal”. In: *arXiv preprint arXiv:0910.5947* (2009).
- [17] Aditi S Krishnapriyan, Maciej Haranczyk, and Dmitriy Morozov. “Topological descriptors help predict guest adsorption in nanoporous materials”. In: *The Journal of Physical Chemistry C* 124.17 (2020), pp. 9360–9368.
- [18] Clément Maria et al. “The gudhi library: Simplicial complexes and persistent homology”. In: *International congress on mathematical software*. Springer. 2014, pp. 167–174.
- [19] Anindya Moitra, Nicholas O Malott, and Philip A Wilsey. “Computation of persistent homology on streaming data using topological data summaries”. In: *Computational Intelligence* 39.5 (2023), pp. 860–899.
- [20] Anindya Moitra, Nicholas O Malott, and Philip A Wilsey. “Persistent homology on streaming data”. In: *2020 International Conference on Data Mining Workshops (ICDMW)*. IEEE. 2020, pp. 636–643.
- [21] Dmitriy Morozov. *Dionysus: Library for computing persistent homology*. C++/Python persistent homology library. 2006. URL: <https://www.mrzv.org/software/dionysus/>.
- [22] Arnur Nigmatov and Dmitriy Morozov. “Distributed computation of persistent cohomology”. In: *2026 Proceedings of the SIAM Symposium on Algorithm Engineering and Experiments (ALENEX)*. SIAM. 2026, pp. 194–206.
- [23] Nina Otter et al. “A roadmap for the computation of persistent homology”. In: *EPJ data science* 6.1 (2017), p. 17.
- [24] Steve Y Oudot. *Persistence theory: from quiver representations to data analysis*. Vol. 209. American Mathematical Society Providence, 2015.
- [25] Pratyush Pranav et al. “The topology of the cosmic web in terms of persistent Betti numbers”. In: *Monthly Notices of the Royal Astronomical Society* 465.4 (2017), pp. 4281–4310.
- [26] Vanessa Robins, Peter John Wood, and Adrian P Sheppard. “Theory and algorithms for constructing discrete Morse complexes from grayscale digital images”. In: *IEEE Transactions on pattern analysis and machine intelligence* 33.8 (2011), pp. 1646–1658.
- [27] Jos BTM Roerdink and Arnold Meijster. “The watershed transform: Definitions, algorithms and parallelization strategies”. In: *Fundamenta informaticae* 41 (2000), pp. 187–228.
- [28] Yashbir Singh et al. “Topological data analysis in medical imaging: current state of the art”. In: *Insights into Imaging* 14.1 (2023), p. 58.

- [29] Floris Takens. “Detecting strange attractors in turbulence”. In: *Dynamical Systems and Turbulence, Warwick 1980* (1981), pp. 366–381.
- [30] Eugene Tan et al. “Selecting embedding delays: An overview of embedding techniques and a new method using persistent homology”. In: *Chaos: An Interdisciplinary Journal of Nonlinear Science* 33.3 (2023).
- [31] Sarah Tymochko et al. “Using persistent homology to quantify a diurnal cycle in hurricanes”. In: *Pattern Recognition Letters* 133 (2020), pp. 137–143.
- [32] Peng Zhang et al. “Near real-time remote sensing based on satellite internet: Architectures, key techniques, and experimental progress”. In: *Aerospace* 11.2 (2024), p. 167.
- [33] Xiaojun Zheng et al. “Percept: a new online change-point detection method using topological data analysis”. In: *Technometrics* 65.2 (2023), pp. 162–178.
- [34] Afra Zomorodian and Gunnar Carlsson. “Computing persistent homology”. In: *Discrete & Computational Geometry* 33.2 (2005), pp. 249–274.